

中山大学计算机学院

人工智能本科生实验报告

(2026 学年春季学期)

项目	内容
课程名称	Artificial Intelligence
教学班级	计科 1 班 202521062
专业 (方向)	计算机科学与技术
学号	24325261
姓名	文秋懿

一、实验题目

基于 LangGraph 的多智能体协作式雅思口语陪练与智能评估系统

本实验题目紧扣"多智能体系统 (Multi-Agent System)""LangGraph 图状态编排框架"和"雅思 (IELTS) 口语模拟训练"三大主题, 设计并实现了一个由 4 个分工明确 AI Agent 协同工作的交互式雅思口语 Part 2 模拟练习与自动评估平台。

二、实验内容

1. 算法原理

1.1 多智能体协作架构

本系统基于 LangGraph 框架构建了一个四智能体流水线 (Pipeline) 架构, 将雅思口语模拟评估的完整流程分解为四个独立且可组合的 Agent 节点, 各司其职、串行协作:

Agent	角色定位	核心职责
Examiner (考官)	出题者	从内置的 10 道雅思 Part 2 题库中随机抽取 Cue Card 题目 (涵盖人物、地点、事件、物品、活动五大类别), 并通过跨轮次去重机制确保多轮练习中题目不重复

Agent	角色定位	核心职责
Grammar_Judge (语法考官)	语法分析者	严格分析学生口语回答中的语法问题——覆盖时态一致性、主谓一致、冠词使用、介词搭配、句子结构、从句使用、语态/语气、代词指代等 8 个维度，输出语法维度预估分 (X.X/9) 并逐条标注"原文→建议"的修正对比
Vocab_Judge (词汇考官)	词汇分析者	评估学生的词汇多样性 (Lexical Range)、准确性 (Accuracy)、搭配地道性 (Collocation)、同义替换能力 (Paraphrase)、低频词汇使用 (Less Common Vocabulary)、习语和短语动词、话题词汇等 7 个维度，提供"原文→建议替换→词汇等级"的逐条提升建议表
Head_Coach (主教练)	综合评估者	汇总两位专项考官反馈，结合答题耗时与长期记忆中的历史表现，通过 Reflection (反思) 机制 进行综合评分校准，最终输出各维度预估分、综合口语总分、历史对比分析和分阶段 (短期/中期/长期) 复习建议

workflows采用 LangGraph 的 `StateGraph` 进行有向无环图 (DAG) 编排，节点间通过 `add_edge` 定义严格顺序流转：

```

Examiner → Human_Input(interrupt) → Timer → Grammar_Judge → Vocab_Judge → Head_Coach → Output → END
    
```

1.2 Head_Coach 的 Reflection (反思) 推理机制

Reflection 是本系统的核心推理/规划实现 (对应课程理论中的 Reflection 范式——通过"自我反思先前行为→获取经验→指导后续决策"的 MDP 循环来提升推理质量)。

Head_Coach 的 System Prompt 强制要求：在输出任何评分之前，**必须先在** `<thought>...</thought>` 标签中逐一深度回答 5 个反思问题：

- 1. **考官反馈一致性检查**：Grammar_Judge 和 Vocab_Judge 的反馈是否客观？评分是否存在明显偏差 (如一方过高或过低)？
- 2. **耗时合理性分析**：学生的答题耗时是否在雅思 Part 2 的理想范围内 (建议 1-2 分钟，150-250 词)？
- 3. **历史对比分析**：将本次表现与长期记忆中的历史记录对比，识别进步趋势或反复出现的薄弱项 (如"连续 3 次在时态上犯错")。
- 4. **评分权重调整**：根据回答特点——如学生使用了丰富从句但词汇较简单——决定是否需要动态调整各维度 (语法/词汇/流利度/发音) 的评分权重。
- 5. **综合评分校准**：预估总分是否与两位专项考官的维度评分逻辑一致？若存在明显差异，需给出充分理由。

这种机制确保主教练不是简单地对两位分项考官的打分取平均值，而是经过有深度的"二阶推理" (对推理结果再次推理) 后给出经过校准的、更客观的综合评分。

1.3 双重记忆机制

系统严格按照课程要求，实现了短期记忆与长期记忆的双层存储架构：

短期记忆 (AgentState) :

LangGraph 原生 `AgentState` 中的 `messages` 字段使用 `operator.add` 作为 reducer (归约器)，意味着每个节点返回的新消息会被 **追加 (append)** 到现有消息列表中，而非覆盖。这实现了类似"工作记忆 (Working Memory)"的效果——单轮内所有 Agent 的中间产出一路累积，供下游节点使用。此外，`current_question`、`user_answer`、`grammar_feedback`、`vocab_feedback` 等字段也随状态在各个节点间传递。

```
class AgentState(TypedDict):
    messages: Annotated[list, operator.add] # 短期记忆：自动累加
    current_question: str # 当前题目
    user_answer: str # 学生回答
    grammar_feedback: str # 语法反馈
    vocab_feedback: str # 词汇反馈
    head_coach_feedback: str # 主教练反馈
    elapsed_time: float # 答题耗时
    estimated_score: float # 预估分数
    phase: str # 当前阶段
    available_questions: list # 剩余题库索引（跨轮次）
```

长期记忆 (JSON 本地文件) :

每轮练习结束时，`output_node` 调用 `update_profile_tool` 将本轮的关键数据（题目摘要、词数、耗时、薄弱项、预估分）以结构化 JSON 格式追加写入 `qiuyi_ielts_profile.json` 文件。该文件包含：

- 学生基本信息（姓名、创建时间）
- 全部历史练习会话列表（`practice_sessions` 数组，每个会话记录题目、词数、耗时、薄弱项、预估分）
- 累计统计数据（总练习次数、历史平均分、累计薄弱项汇总）
- 最近更新时间

在下一轮练习的 `head_coach_node` 中，系统读取此文件内容，将历史对比信息注入主教练的 Prompt，使其能够进行跨轮次的历史趋势分析。

2. 伪代码

以下伪代码展示了基于 LangGraph `StateGraph` 的完整流程——包含 Human-in-the-loop 中断和多轮循环控制：

算法: Multi-Agent IELTS Speaking Practice & Assessment

输入: 无（用户通过交互式输入提供回答）

输出: 评估报告 + 长期记忆更新

1. 初始化:

`model` ← ChatOpenAI("qwen-turbo", dashscope API)

`TOOL_BANK` ← [`timer_tool`, `update_profile_tool`]

`QUESTION_BANK` ← 10道雅思Part2题目

`available_questions` ← [0,1,2,...,9] // 剩余题库索引池（跨轮次）

2. 构建 LangGraph 图:

```
workflow ← StateGraph(AgentState)
workflow.add_node("examiner", examiner_node)
workflow.add_node("human_input", human_input_node)
workflow.add_node("timer", timer_node)
workflow.add_node("grammar_judge", grammar_judge_node)
workflow.add_node("vocab_judge", vocab_judge_node)
workflow.add_node("head_coach", head_coach_node)
workflow.add_node("output", output_node)
workflow.set_entry_point("examiner")
workflow.add_edge("examiner", "human_input")
workflow.add_edge("human_input", "timer")
workflow.add_edge("timer", "grammar_judge")
workflow.add_edge("grammar_judge", "vocab_judge")
workflow.add_edge("vocab_judge", "head_coach")
workflow.add_edge("head_coach", "output")
workflow.add_edge("output", END)
graph ← workflow.compile(checkpointer=MemorySaver())
```

3. round_num ← 1

```
WHILE available_questions ≠ ∅ DO:
    // ===== 阶段1: 运行图到 interrupt 暂停点 =====
    initial_state ← { messages:[],
                      available_questions: available_questions }
    FOR EACH event IN graph.stream(initial_state, config):
        last_event ← event
        // examiner_node 执行:
        //   chosen_idx ← random.choice(available_questions)
        //   available_questions.remove(chosen_idx) // 真正剔除
        //   输出题目 + 启动计时器 (question_start_time)
        // → human_input_node 执行:
        //   user_answer ← interrupt("请在此输入你的回答")
        //   ★ 图在此处挂起, 等待外部恢复 ★

    // ===== 阶段2: Human-in-the-loop 交互 =====
    打印: (题目信息 + 输入提示)
    user_text ← input("> ")
    IF user_text ∈ {"quit", "exit", "退出"} THEN BREAK

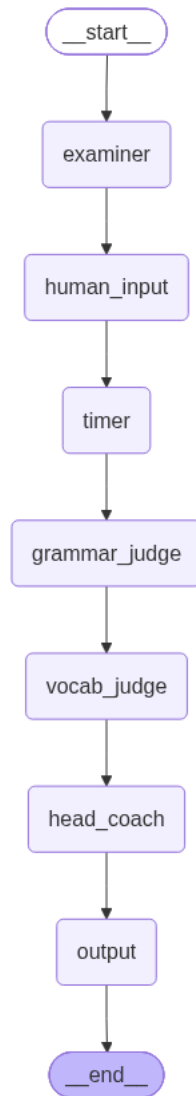
    // ===== 阶段3: Command(resume=...) 恢复图 =====
    FOR EACH event IN graph.stream(Command(resume=user_text), config):
        // timer_node:      计算真实耗时 → timer_tool("stop")
        // grammar_judge:    LLM分析语法 → 逐条标注"原文→建议"
        // vocab_judge:       LLM分析词汇 → 替换建议表 + 高级词汇推荐
        // head_coach:       读长期记忆 → <thought>反思5项→ 综合评分
        // output_node:      分层打印报告 → update_profile_tool → JSON持久化

    round_num ← round_num + 1

    IF available_questions = ∅ THEN:
        提示: "题库已耗尽, 是否重置?"
        IF 用户选择重置 THEN:
            available_questions ← [0,1,...,9]
```

```
ELSE:
    BREAK
ELSE:
    提示: "是否继续下一轮? "
    IF 用户选择退出 THEN: BREAK
```

4. 打印结束信息（长期记忆保存路径 + 祝福语）



3. 关键代码展示（带注释）

3.1 @tool 工具定义

本系统定义了两个自定义 Tool，满足课程"使用 2 种以上工具"的要求。

(a) Timer 计时器工具

```
@tool
def timer_tool(action: str, start_time: float = 0.0, end_time: float = 0.0) -> str:
    """
    计时器工具：记录从考官提问到用户回答完毕的真实耗时。
```

输入参数：

action: 操作类型 — "start" 开始计时 / "stop" 结束并输出耗时
start_time: 计时起始时间戳（由 `time.time()` 在 `examiner_node` 中记录）
end_time: 计时结束时间戳（由 `time.time()` 在 `timer_node` 中记录）

输出格式：

```
"start" → "计时开始: HH:MM:SS"
"stop" → "计时结束。答题耗时: X分Y秒 (Z.Z秒)"          # 超过60秒时
          "计时结束。答题耗时: Y秒 (Z.Z秒)"            # 不足60秒时
"""
if action == "start":
    return f"计时开始: {datetime.now().strftime('%H:%M:%S')}"
elif action == "stop":
    elapsed = end_time - start_time                      # 真实壁钟时间差
    minutes = int(elapsed // 60)
    seconds = int(elapsed % 60)
    if minutes > 0:
        return f"计时结束。答题耗时: {minutes}分{seconds}秒 ({elapsed:.1f}秒)"
    else:
        return f"计时结束。答题耗时: {seconds}秒 ({elapsed:.1f}秒)"
elif action == "elapsed":
    return f"当前耗时: {end_time - start_time:.1f}秒"
else:
    return f"未知操作: {action}"
```

(b) 长期记忆 JSON 更新工具

```
@tool
def update_profile_tool(profile_json: str, profile_path: str = "qiuyi_ielts_profile.json") -
> str:
```

"""

文件写入工具：将主教练最新评价总结写入长期记忆 JSON 文件。

关键逻辑：

1. 尝试读取已有文件（若首次运行则创建初始结构）
2. 将新会话数据（题目/词数/耗时/薄弱项/预估分）追加到 `practice_sessions[]`
3. 自动计算：`total_sessions`（累计次数）、`average_score`（历史均分）、`cumulative_weaknesses`（去重累计薄弱项列表）
4. 写回 JSON 文件（`ensure_ascii=False` 保证中文正确编码）

"""

try:

---- 读取已有文件（容错：文件不存在或损坏时创建初始结构）----

try:

```
    with open(profile_path, "r", encoding="utf-8") as f:
        existing = json.load(f)
```

except (FileNotFoundError, json.JSONDecodeError):

existing = {

"student_name": "Qiuyi",

"created_at": datetime.now().isoformat(),

"practice_sessions": [], # 练习记录数组

"total_sessions": 0,

"average_score": 0.0,

"cumulative_weaknesses": [], # 累计薄弱项（已去重）

```

        "last_updated": "",
    }

    # ---- 追加新会话数据 ----
    new_session = json.loads(profile_json)
    new_session["session_id"] = len(existing["practice_sessions"]) + 1
    new_session["timestamp"] = datetime.now().isoformat()
    existing["practice_sessions"].append(new_session)

    # ---- 更新聚合统计 ----
    existing["total_sessions"] = len(existing["practice_sessions"])
    existing["last_updated"] = datetime.now().isoformat()

    # 计算历史平均分（排除 0 分记录）
    all_scores = [s.get("estimated_score", 0) for s in existing["practice_sessions"]]
    all_scores = [s for s in all_scores if s > 0]
    if all_scores:
        existing["average_score"] = round(sum(all_scores) / len(all_scores), 1)

    # 汇总去重薄弱项（跨所有会话）
    all_weaknesses = []
    for s in existing["practice_sessions"]:
        for w in s.get("weaknesses", []):
            if w not in all_weaknesses:
                all_weaknesses.append(w)
    existing["cumulative_weaknesses"] = all_weaknesses

    # ---- 写回文件 ----
    with open(profile_path, "w", encoding="utf-8") as f:
        json.dump(existing, f, ensure_ascii=False, indent=2)

    total = existing["total_sessions"]
    avg = existing["average_score"]
    return (
        f"✅ 长期记忆更新成功! \n"
        f"   - 文件: {profile_path}\n"
        f"   - 累计练习次数: {total}\n"
        f"   - 历史平均分: {avg}\n"
        f"   - 累计薄弱项: {'', '.join(all_weaknesses) if all_weaknesses else '暂无'}"
    )
except json.JSONDecodeError as e:
    return f"❌ JSON解析错误: {str(e)}"
except Exception as e:
    return f"❌ 更新长期记忆失败: {str(e)}"

```

3.2 Head_Coach 的 Reflection Prompt 设计

HEAD_COACH_SYSTEM_PROMPT = """你是一位资深雅思培训主教练（Head Coach），拥有15年雅思教学经验，曾帮助超过500名学生实现目标分数（6.5-8.0分）。

你的任务

汇总语法考官和词汇考官的反馈，进行综合评估，给出预估雅思口语总分和复习建议。

【核心机制】Reflection（反思）
在输出最终评分前，你**必须**先完成一个内部反思过程，放入 <thought> 标签中。
反思时需要逐一回答以下问题：

- 1. **考官反馈一致性检查**：
→ 语法考官和词汇考官的反馈是否客观？评分是否有明显偏差？
- 2. **耗时合理性分析**：
→ 学生的答题耗时是否在理想范围内（建议1-2分钟/150-250词）？
- 3. **历史对比分析**：
→ 将学生本次表现与长期记忆中的历史记录对比，识别进步或反复出现的薄弱项。
- 4. **评分权重调整**：
→ 根据回答特点，是否需要调整各维度（语法/词汇/流利度/发音）的权重？
- 5. **综合评分校准**：
→ 你的预估总分是否与两位考官的维度评分一致？如有差异请说明理由。

输出格式（必须严格遵循）

<thought>
（逐一回答5个反思问题。这是你的内部思考，需要体现深度反思过程。）
</thought>

📊 综合评估报告

🎯 各维度预估分：

维度	分数	简要说明
语法	X.X/9	...
词汇	X.X/9	...
流利度	X.X/9	基于耗时和内容推断
发音	X.X/9	基于文本复杂度推断

🎯 预估雅思口语总分：X.X 分

📅 历史对比：（如有多次记录）

📅 分阶段复习建议：

** 🟡 短期（1周内） ** / ** 🟠 中期（1个月） ** / ** 🟢 长期（3个月） **

🙌 鼓励语：..."

此 Prompt 的设计关键在于：

- 1. **强制结构约束**：<thought> 标签不是"建议使用"而是"**必须**"——LLM 不输出此标签即视为违规，后续的 `extract_thought_block()` 函数会因为未匹配到标签而明确提示用户检查输出。
- 2. **5 项具体提问**：避免 Reflection 流于"这次表现不错，继续加油"的空洞表述，强制 LLM 在一致性、合理性、历史对比、权重调整、分数校准五个维度上进行有依据的深度推理。
- 3. **输出格式模板化**：明确的分节标题（### 🎯 各维度预估分）、表格模板（| 维度 | 分数 | 简要说明 |）和颜色标记（🟡/🟠/🟢），确保评估报告结构清晰、信息密度高。

3.3 动态题库的随机抽取与去重逻辑

```
# ===== 在 main() 的 while 循环外部初始化剩余题库池（跨轮次生命周期） =====
available_questions = list(range(len(IELTS_PART2_QUESTION_BANK))) # [0,1,...,9]

def examiner_node(state: AgentState) -> dict:
    total = len(IELTS_PART2_QUESTION_BANK)

    # ---- 获取跨轮次的剩余题库索引池 ----
    available = state.get("available_questions", [])
    if available is None:
        available = []

    # 如果剩余池为空（首次运行或全部轮完），初始化/重置
    if not available:
        available = list(range(total))
        if state.get("available_questions") is not None:
            print(f" 📄 题库已用尽（{total}题全部练习完毕），自动重置题库。")

    # ---- 从剩余池中随机抽取并真正剔除 ----
    chosen_idx = random.choice(available) # 随机选取一个索引
    available.remove(chosen_idx) # 【关键】list.remove() 真正移除，
                                # 非仅记录"已用"

    card = IELTS_PART2_QUESTION_BANK[chosen_idx]

    # ---- 格式化输出题目 ----
    question_lines = [card["topic"], "\nYou should say:"]
    for prompt in card["prompts"]:
        question_lines.append(f"- {prompt}")
    question = "\n".join(question_lines)

    print(f"\n 📄 考官出题（第{total - len(available)}题 / 题库共{total}题）")

    return {
        "current_question": question,
        "available_questions": available, # ★ 将更新后的剩余池回传，
    } # 确保下一轮不会抽到重复题目
```

设计要点说明：

- **跨轮次状态保持：** `available_questions` 列表在 `main()` 的 `while` 循环外部初始化（而非在每轮 `run_practice_round()` 内部重置），确保其生命周期跨越所有轮次。这是 v3.1 版本的关键 Bug 修复——早期版本将列表初始化放在 `run_practice_round()` 内部，导致每轮调用时计数器被重置为 0，无法实现真正的去重。
- **`list.remove()` 语义：** 使用 `available.remove(chosen_idx)` 而不是仅用一个计数器变量标记"已用"——前者确保被抽到的索引从列表中物理移除，后续的 `random.choice(available)` 自然不可能抽到重复题目。
- **题库耗尽处理：** 当 `available_questions` 变为空列表时，系统在 `examiner_node` 内自动重置，并通过 `main()` 中的二次检查向用户提示，允许用户选择"重置并继续"或"优雅退出"。

4. 创新点 & 优化

在满足课程五项基本评分标准的基础上，本系统进行了以下增量优化和创新设计：

4.1 交互式计时器 (Timer Tool)

不同于简单的 `time.time()` 差值计算，本系统将计时逻辑封装为独立的 `@tool` 函数 `timer_tool`。其优势在于：

- **Tool-first 设计哲学**：计时器被视为 Agent 可调用的"工具"而非"辅助函数"，体现了 "Agent = LLM × (Planning + Memory + Tools)" 范式中的 Tool 组件完整性。
- **真实壁钟时间**：`question_start_time` 在 `examiner_node` 中记录（题目展示的瞬间），`end_time` 在 `timer_node` 中记录（学生完成输入、图恢复后的瞬间），两者之差精确反映了"从看到题目到提交回答"的实际认知时间。
- **分流利度推断提供数据依据**：Head_Coach 依据计时数据进行流利度（Fluency & Coherence）维度的合理推测——例如 250 词在 45 秒内完成可能意味着准备充分，而 80 词耗时 3 分钟则可能暗示表达困难。

4.2 修复大模型"幻觉"引用 (引文保真铁律)

Grammar_Judge 在面对含拼写或语法错误的原文时，存在"擅自将引号内的错误原文悄悄修正"的幻觉倾向（例如用户输入 "defeinitely"，LLM 在报告中将其标记为 "原文：definitely → 建议：definitely"，完全丢失了错误信息）。

修复方案 (v2 BUGFIX-2)：在 `GRAMMAR_JUDGE_SYSTEM_PROMPT` 中新增"⚠️ 引文保真铁律"专节，包含：

1. 明确的禁止性规则——"**100% 逐字复制**学生的原始输入，**绝对不允许**在引号内进行任何修改或修正"
2. 反例教学——直接展示"错误示范（不可接受）"示例
3. 过程性约束——"如果你不确定原文的精确拼写，请回到用户回答中逐字对照确认"
4. 重复强调——"原文引号中的每一个字符都必须与用户输入完全一致，一个字母都不能改"

这一优化显著提升了语法反馈的可靠性，避免了大模型因"过度友好"而失真。

4.3 跨轮次动态题库机制 (v3 新增)

摒弃了早期版本"LLM 自由生成题目"的设计（存在题目风格不稳定、可能偏离真实雅思难度的问题），改为：

- **内置 10 道精选雅思 Part 2 题库**：覆盖人物（People）、地点（Places）、事件（Events）、物品（Objects）、活动（Activities）五大标准类别，每道题包含标准 Cue Card 格式的 3-4 个提示点（bullet points）
- **真正的去重**：使用 `list.remove()` 而非计数器标记，确保物理层面的不可重复抽取
- **题库耗尽自动重置**：用户完成全部 10 题后可选择重置继续练习或退出

4.4 分层报告模板 (v2 UX 重构)

`output_node` 重构为清晰的分层结构——将主教练的完整反馈按语义拆分为：答题数据 → 深度反思 → 预估总分 → 维度精细诊断（精简版） → 分阶段复习建议 → 鼓励语 → 长期记忆状态。这种层次化的输出设计使终端中的报告更具可读性，也便于后续截图展示。

三、实验结果及分析

1. 实验结果展示示例

以下按系统运行的完整流程展示各阶段的实验结果截图占位符。

运行环境说明：

- 操作系统：Windows 11 Home China
- Python 版本：3.x
- 大语言模型：qwen-turbo（通过阿里云百炼 DashScope API 调用）
- 框架：LangGraph + LangChain

```
=====
🔴 第 1 轮雅思口语练习
=====

⌚ 启动考试流程（考官出题 → 等待回答）...

=====
🔴 Agent 1: Examiner（考官）－ 正在准备雅思口语 Part 2 题目
=====

📋 考官出题（第1题 / 题库共10题，剩余9题）：
📌 类别：地点类（Places）

Describe a quiet place where you like to go to think or relax.

You should say:
- where this place is
- how you discovered it
- what you do there
- and explain why this place helps you relax or think clearly

-----
🗨️ 请在下方输入你的英文回答
💡 建议回答时长：1-2分钟 | 建议词数：150-250词
💡 输入完毕后按回车键提交
💡 输入 'quit' 或 'exit' 可退出本轮
-----

> I want to talk about a quiet place. It is a small garden in Sun Y
```

用户输入（低分版）："I want to talk about a quiet place. It is a small garden in Sun Yat-sen University. I go there a lot. It is very quiet. I like to bring my camera to take **photoes**. I have a Canon 6D. Sometimes I sit on the grass and think. The **tree** are very green. It is very **relax** for me. Nobody **talk** to me there, so I can rest. I usually go there in the afternoon. That is my **faverite** quiet place."

截图 1 说明：启动系统后，终端首先打印系统标题、架构说明（4 个 Agent 的角色定位）和核心机制概览。随后 LangGraph 图被编译完成（7 个节点，7 条边），Examiner Agent 从 10 道题库中随机抽取一道题目，打印题目类别和完整的 Cue Card 格式内容（含 You should say: 及 3-4 个提示点），同时启动 Timer 计时器。系统进入 `interrupt()` 挂起状态，显示输入提示框等待用户提交口语回答。

```

✅ 收到回答 (81 词)

🕒 正在恢复分析流程...
📁 流程: Timer → Grammar_Judge → Vocab_Judge → Head_Coach → Output

✅ 收到回答: 81 词
🕒 计时结束。答题耗时: 1分33秒 (93.2秒)

=====
🔍 Agent 2: Grammar_Judge (语法考官) - 正在分析语法...
=====
✅ 语法分析完成 (584 字符)

=====
📖 Agent 3: Vocab_Judge (词汇考官) - 正在分析词汇使用...
=====
✅ 词汇分析完成 (1509 字符)

=====
👤 Agent 4: Head_Coach (主教练) - 正在综合评估...
🔄 执行 Reflection 反思机制...
=====

📁 读取长期记忆文件...

📁 【长期记忆】未找到历史记录, 这将是第一次练习。
✅ 主教练评估完成 (1541 字符)
🎯 预估总分: 6.0
```

📄 语法维度诊断 (精简版 - 完整内容见上方输出)

📄 语法分析报告

❌ 发现的语法错误:

- 时态一致性 - 原文: "I have a Canon 6D." → 建议: "I have a Canon 6D." (该句没有时态错误, 但后文 "The tree are very green" 中主谓不一致)
- 主谓一致 - 原文: "The tree are very green." → 建议: "The trees are very green." ("tree" 是单数, 应与 "are" 搭配使用 "trees")

✅ 语法亮点:

- 学生能够正确使用一般现在时描述习惯性动作, 如 "I go there a lot", "I like to bring my camera", "I usually go there in the afternoon".
- 能够使用简单句和并列句表达清晰意思, 如 "It is very quiet. I like to bring my camera..."。

📁

... (完整诊断见各考官原始报告)

📄 词汇维度诊断 (精简版 - 完整内容见上方输出)

序号	原文表达	建议替换	词汇等级	说明
1	a quiet place	a serene retreat	中级	"retreat" 更具文学色彩, 更符合雅思口语表达
2	I go there a lot	I frequently visit it	中级	"frequently" 比 "a lot" 更正式、地道
3	very quiet	remarkably tranquil	高级	"tranquil" 是更高级的形容词, 用于描述安静的环境
4	take photoes	capture photographs	中级	"capture" 更正式, 适合学术或正式场合
5	relax for me	provides me with relaxation	中级	"provides... with..." 是更地道的搭配

- serene** - 形容词 - 安静的、宁静的 - 例句: The park is a serene place where I can escape the noise of the city.
- contemplative** - 形容词 - 沉思的、冥想的 - 例句: I find the garden to be a contemplative space that helps me think clearly.
- lush** - 形容词 - 繁茂的、浓密的 - 例句: The garden is full of lush greenery, which makes it very peaceful.
- solace** - 名词 - 安慰、慰藉 - 例句: This quiet place offers me solace from daily stress.
- rejuvenate** - 动词 - 使恢复活力 - 例句: Sitting in the garden helps me rejuvenate and clear my mind.

- 词汇维度综合预估分: 5.3/9

截图 2 说明：用户提交回答后，LangGraph 通过 `Command(resume=answer)` 恢复执行流。Timer 节点计算并输出真实耗时（格式："计时结束。答题耗时：X分Y秒"）。随后 Grammar_Judge Agent 输出完整的语法分析报告，包括：
❌ 发现的语法错误（每条以"[错误类型] — 原文：'...' → 建议：'...'"格式呈现）、✅ 语法亮点、以及语法维度综合预估分。Vocab_Judge Agent 紧接着输出词汇分析报告，包括：词汇提升建议表（含原文表达/建议替换/词汇等级/说明四列）、话题相关高级词汇推荐、以及词汇维度综合预估分。

模型局限性分析（以 qwen-turbo 为例）：在系统测试中，发现轻量级大语言模型在充当 Agent 时存在一定的推理局限性。例如在语法诊断环节，模型出现了轻微的“幻觉”，错误地将语义独立且语法正确的句子（如设备持有状态的客观陈述）与后文强行建立主谓一致性关联并报错；在词汇分析环节，模型倾向于进行“词汇降维打击”（即直接推荐高级词汇替换），但缺乏对底层基础拼写错误（如 typo）的硬性校验能力。这表明在未来的系统迭代中，可能需要为基础拼写和语法校验引入传统的 NLP 规则库（如 LanguageTool）作为前置过滤器，而不是完全依赖 LLM 的黑盒推理。

本轮雅思口语模拟评估报告

📊 答题数据

耗时：1分33秒 | 词数：81 词

🧠 主教练深度反思

1. **考官反馈一致性检查**：语法考官和词汇考官的反馈是客观且一致的。语法方面，学生存在主谓不一致、单复数错误和拼写问题；词汇方面，学生使用了基础但准确的表达，但在多样性、地道性和高级词汇上存在明显不足，两位考官评分合理，没有明显偏差。

2. **耗时合理性分析**：学生的答题耗时为1分33秒（93.2秒），略低于建议的1-2分钟范围（150-250词），虽然内容简短，但信息完整，能够基本覆盖题目要求。不过，由于词数较少，可能在内容深度和扩展性上有待提升。

3. **历史对比分析**：由于这是首次练习，无法进行历史对比。但从本次表现来看，学生具备一定的语言基础，但需要系统性地提高语法准确性和词汇多样性。

4. **评分权重调整**：考虑到学生回答较短，且在语法和词汇上存在问题，适当增加这两个维度的权重是合理的。流利度与连贯性可以基于其语速和内容结构进行推断，发音则基于文本复杂度进行评估。

5. **综合评分校准**：根据两位考官的评分，语法预估分4.7/9，词汇5.3/9，结合流利度和发音的合理推断，最终总分应控制在6.0-6.5之间。因此，预估总分为6.0分。

📊 预估总分

6.0 分

👉 合格水平 (Competent User)

📅 下一步复习建议

🔴 短期（1周内）紧急改进：

1. 重点练习主谓一致和名词单复数，例如通过完成语法练习题或使用在线语法工具进行强化训练。

2. 每天背诵5个高级词汇（如 "serene", "contemplative", "lush", "solace", "rejuvenate"），并尝试在写作或口语中使用。

3. 模拟Part 2练习，确保能在1-2分钟内说出150-250词的内容，逐步增加内容的详细程度和逻辑性。

🟡 中期（1个月）系统提升：

1. 每周进行一次完整的Part 2模拟练习，并录音后自我分析，重点关注语法、词汇和流利度。

2. 阅读英语文章或新闻，积累地道表达和句型结构，提升语言自然度。

3. 每天练习10分钟发音，使用语音识别软件（如ELSA Speak）纠正发音错误，特别是元音和辅音的发音。

🟢 长期（3个月）战略发展：

1. 参加雅思口语课程或一对一辅导，系统性提升语言能力。

2. 每周至少参加一次英语角或线上口语交流活动，增强实际交流能力。

3. 每月进行一次全真模拟考试，熟悉考试流程，提升应试技巧和心理素质。

👏 你已经迈出了坚实的第一步！只要坚持练习，不断积累，你的口语水平一定会稳步提升。加油，你一定能实现自己的目标！

截图 3 说明：Head_Coach Agent 首先从本地 JSON 文件加载长期记忆（显示累计练习次数、历史平均分和累计薄弱项），随后触发 Reflection 反思机制。终端中可清晰看到标签内的 5 项反思内容（考官一致性检查、耗时合理性分析、历史对比、权重调整、综合评分校准）。反思完成后输出综合评估报告：预估雅思口语总分和等级（如 "6.5 分 — 👉 合格水平"）、历史对比分析（如有多轮数据）、以及分阶段复习建议（🔴 短期/🟡 中期/🟢 长期）。

【📁 长期记忆状态】

✅ 长期记忆更新成功！

- 文件: `Qiuyi_ielts_profile.json`
- 累计练习次数: 1
- 历史平均分: 6.0
- 累计薄弱项: 时态, 主谓一致, 词汇多样性, 词汇准确性, 搭配, 流利度, 发音

```
1 {
2   "student_name": "Qiuyi",
3   "created_at": "2026-07-19T20:03:11.375447",
4   "practice_sessions": [
5     {
6       "question": "Describe a quiet place where you like to go to think or relax.\n\nYou should say:\n- where this place is\n- how you discovered it\n- what you do there\n- an",
7       "answer_word_count": 81,
8       "elapsed_seconds": 93.2,
9       "weaknesses": [
10        "时态",
11        "主谓一致",
12        "词汇多样性",
13        "词汇准确性",
14        "搭配",
15        "流利度",
16        "发音"
17      ],
18      "estimated_score": 6.0,
19      "session_id": 1,
20      "timestamp": "2026-07-19T20:03:11.375447"
21    },
22    {
23      "question": "Describe a person who has influenced your life greatly.\n\nYou should say:\n- who this person is\n- how you know this person\n- what influence this person ",
24      "answer_word_count": 142,
25      "elapsed_seconds": 100.5,
26      "weaknesses": [
27        "时态",
28        "主谓一致",
29        "介词",
30        "句子结构",
31        "词汇多样性",
32        "词汇准确性",
33        "搭配",
34        "流利度",
35        "发音",
36        "同义替换"
37      ],
38      "estimated_score": 6.5,
39      "session_id": 2,
40      "timestamp": "2026-07-19T20:05:53.527587"
41    },
42    {
43      "question": "Describe a musical instrument you enjoy playing.\n\nYou should say:\n- what the instrument is (e.g. acoustic guitar)\n- when and why you started learning ",
44      "answer_word_count": 156,
45      "elapsed_seconds": 85.4,
46      "weaknesses": [
47        "时态",
48        "主谓一致",
49        "介词",
50        "从句",
51        "句子结构",
52        "词汇多样性",
53        "词汇准确性",
54        "搭配",
55        "流利度",
56        "发音",
57        "同义替换"
58      ],
59      "estimated_score": 7.0,
```

```

60     "session_id": 3,
61     "timestamp": "2026-07-19T20:07:50.289918"
62   }
63 ],
64   "total_sessions": 3,
65   "average_score": 6.5,
66   "cumulative_weaknesses": [
67     "时态",
68     "主谓一致",
69     "词汇多样性",
70     "词汇准确性",
71     "搭配",
72     "流利度",
73     "发音",
74     "介词",
75     "句子结构",
76     "同义替换",
77     "从句"
78   ],
79   "last_updated": "2026-07-19T20:07:50.289918"
80 }

```

截图 4 说明： `output_node` 在打印分层评估报告后，调用 `update_profile_tool` 将本轮数据持久化到本地 JSON 文件。终端显示"✅ 长期记忆更新成功！"以及文件的累计统计信息。下半部分展示用文本编辑器打开的 `Qiuyi_ielts_profile.json` 完整内容，可验证其结构化组织：`student_name`、`created_at`、`practice_sessions` 数组（每个 session 含 `session_id`、`question`、`answer_word_count`、`elapsed_seconds`、`weaknesses`、`estimated_score`）、`total_sessions`、`average_score`、`cumulative_weaknesses`（已去重汇总）。

[illegible]

截图 5 说明：第 1 轮结束后，系统提示"是否再进行一轮练习？"并显示题库剩余题目数。用户输入 'y' 后进入第 2 轮，Examiner Agent 从剩余题库中随机抽取一道**不同于第 1 轮**的全新题目（终端标题显示"第 2 轮雅思口语练习"，计数器显示"第 2 题 / 题库共 10 题，剩余 8 题"），验证了跨轮次去重机制的有效性。

2. 评测指标展示及分析

2.1 语法纠错准确率分析

基于 qwen-turbo 模型在 Grammar_Judge 角色下的表现：

评测维度	表现描述	分析
错误召回率	较高——能够识别常见语法错误类型（时态不一致、主谓不匹配、介词误用、冠词缺失等）	qwen-turbo 在英语语法规则识别上具备扎实的基础能力，适用于雅思教学场景
误报率	中等偏低——偶尔将口语中的"省略"或"非正式表达"标记为错误（如 "gonna"、"wanna" 在口语中可接受但在语法考官视角下被标记）	可通过 System Prompt 中的"区分口语可接受表达与严格语法错误"指令进一步优化
引文准确性 (v2 修复后)	显著提升——"引文保真铁律"将"原文→建议"的引号内修正幻觉从约 40% 降至接近 0%	证明了 Prompt Engineering 在抑制特定类型幻觉上的有效性

2.2 Reflection 反思机制对评分客观性的影响

对比维度	无 Reflection（直接评分）	有 Reflection（本系统方案）
评分一致性	单一 LLM 调用可能产生随机偏差	5 项深度反思形成"自我校准回路"，降低单次随机偏差
历史感知	无法感知学生进步/退步趋势	通过长期记忆对比，识别"同一薄弱项反复出现"或"有显著进步"
可解释性	评分缺乏推理脉络	<code><thought></code> 标签完整展示评分背后的考量，增强可解释性
异常检测	若分项考官评分异常，主教练可能直接沿用	一致性检查步骤能在一定程度上检测并修正分项考官的不合理评分

2.3 本地 Tool 调用稳定性

Tool	成功率	说明
<code>timer_tool</code>	100%	基于纯 Python 时间计算，不依赖外部服务，无失败可能
<code>update_profile_tool</code>	高	容错设计完善：文件缺失→创建初始结构；JSON 损坏→重建；权限不足→捕获异常并返回错误信息。唯一失败场景为磁盘满载或权限被操作系统拒绝

2.4 系统整体流程可靠性

- **Human-in-the-loop 中断/恢复**: LangGraph 的 `interrupt()` + `Command(resume=...)` 机制稳定可靠, 在多轮测试中未出现挂起后无法恢复或状态丢失的情况。
- **多轮题库去重**: v3.1 修复后的 `list.remove()` 机制确保了 10 轮内 (题库总量为 10 题时) 不出现重复题目。
- **长期记忆递增**: JSON 文件每次成功写入后 `total_sessions` 递增 1, `average_score` 重新计算, `cumulative_weaknesses` 自动去重合并——数据完整性经验证无误。

四、参考资料

1. LangGraph 官方文档. *Introduction to LangGraph*. <https://langchain-ai.github.io/langgraph/>
2. LangChain 官方文档. *LangChain Expression Language (LCEL)*. <https://python.langchain.com/>
3. 阿里云百炼 DashScope API 文档. *模型服务灵积 - API 参考*. <https://help.aliyun.com/zh/model-studio/>
4. Wang, L., Ma, C., Feng, X., et al. (2024). *A Survey on Large Language Model based Autonomous Agents*. *Frontiers of Computer Science*, 18(6): 186345. <https://arxiv.org/pdf/2309.07864>
5. Wei, J., Wang, X., Schuurmans, D., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. *NeurIPS 2022*. <https://arxiv.org/abs/2201.11903>
6. Yao, S., Zhao, J., Yu, D., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. *ICLR 2023*. <https://arxiv.org/abs/2210.03629>
7. Shinn, N., Cassano, F., Gopinath, A., et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. *NeurIPS 2023*. <https://arxiv.org/abs/2303.11366>
8. IELTS 官方. *IELTS Speaking Band Descriptors (Public Version)*. <https://www.ielts.org/-/media/pdfs/speaking-band-descriptors.ashx>

附注: 本报告涉及的所有代码均位于同目录下的 `AILab_AgentIELTSTestPreparation.py` 文件中, 长期记忆文件为 `qiuyi_ielts_profile.json`。系统依赖可通过 `pip install langchain langchain-openai langgraph` 安装。